QUANTUM HACKATHON

Production Scheduling Optimization (ERP)

Challenge: *Optimize a factory production schedule and solve the core sequencing problem on a quantum simulator and a QUBO solver.*

1. Introduction

MetalTech Ltd. manufactures metal components for industrial customers. The production plant has three machines: a ***lathe (T)****, a* ***milling machine (F)****, and a* ***welding station (S)****. Every part passes through the machines in a strict order*

T → F → S: first turning, then milling, then welding

and while one part occupies a machine, no other part can use it.

On Friday afternoon, just before the weekend, an urgent order from a key customer lands in the inbox: three different parts (jobs Z1, Z2 and Z3), each with different processing times on each machine, all to be finished as soon as possible. The production manager, Mr. Zenon, must fix the order and schedule so that the whole batch is finished as early as possible.

Your team receives the data: the jobs, the machines, the known processing times, and the fixed technological order T → F → S. **Your job is to find the schedule that minimizes the completion time of the whole batch (the makespan)**.

Your task: build software that optimizes the production plan to minimize the overall makespan of all jobs. The core search for the optimal arrangement of operations on machines must be implemented **as a quantum problem** and solved on a **simulator** or on a **QUBO solver**.

2. Problem Definition

Each job consists of operations processed by every machine exactly once, in the fixed order T → F → S. Every operation has a known duration. The objective is to minimize the completion time of the last operation of the last job (the makespan), subject to:

- each operation starts exactly once,
- no machine processes two operations at the same time,
- operations within a single job respect the order T → F → S.

This is a classic **Job-Shop Scheduling Problem (JSSP)** in its flow-shop form (all jobs share the same machine order). For two and three machines an optimal schedule always exists in which every machine processes the jobs in the same order (a permutation schedule), which justifies sequence-based encodings.

3. How the Challenge Works

The challenge has two stages. Read this section carefully — it is the key to knowing how you will be scored.

Stage 1 — Mandatory baseline (Section 4). A fixed, well-defined problem that every team must solve. It is deliberately small and standard: today it is solvable with essentially one good AI prompt, so everyone will land at a similar level here and we do not differentiate teams on it. It is the entry ticket — without a working baseline, a team is not assessed further.

Stage 2 — Your own extension (Section 5). Starting from the baseline, you define and develop your own, more ambitious problem, combining any of the directions listed in Section 5 (or proposing your own within the manufacturing/scheduling domain). This is the main source of points, not a bonus. Depth, quality, and your own inventiveness are what set your team apart.

> *In short: the baseline proves you can do it; the extension proves you understand it and can take it somewhere new.*

4. Stage 1 — Mandatory Baseline

You must deliver a working reference solution for both instances below. The reference makespans are given so you can verify your own results.

4.1 Instance A — 2 jobs × 2 machines

A reduced two-machine version (lathe **T**, milling **F**).

Job	Lathe (T) [h]	Milling (F) [h]
Z1	2	1
Z2	1	2

- **Order:** T → F.
- **Required:** the QUBO for the 2×2 instance solved with a quantum algorithm, either on quantum hardware or on a simulator, returning a feasible schedule.

4.2 Instance B — 3 jobs × 3 machines

The full MetalTech scenario: three machines (lathe **T**, milling **F**, welding **S**).

Job	Lathe (T) [h]	Milling (F) [h]	Welding (S) [h]
Z1	2	1	2
Z2	1	2	1
Z3	2	2	1

- **Order:** T → F → S.
- **Required:** the QUBO for the 3×3 instance solved on any QUBO solver (quantum, hybrid or a classical solver), returning a feasible schedule. The quantum or hybrid solvers are preferred and may increase the submission's score in the "Quality of the solution" criterion.

4.3 Why these two sizes

The two instances exist to separate the quantum method from the scale: Instance A lets you actually run and inspect your solution; Instance B forces you to confront the qubit blow-up of the time-indexed formulation and to reach for a solver that handles it.

5. Stage 2 — Problem Extension

Starting from the baseline, take the problem further: make the factory model more realistic, and make its quantum solution as compact, accurate, and well understood as you can. Every real-world constraint you add costs variables and

penalty terms, so the challenge is to model more of the factory while spending fewer qubits, and to show the result is still correct. The highest scores go to teams that combine the richest model with the smallest correct formulation, the best-justified algorithm, and the most rigorous analysis.

Examples of extension directions - combine them freely, or propose your own within a realistic factory/scheduling context:

- **Qubit reduction** — how far can you shrink the QUBO for the 3×3 instance (72 variables in the plain time-indexed formulation at H = 8) without losing correctness, and at what cost? (approximations, decomposition, partitioning, alternative encodings such as log/binary time encoding, permutation-based encodings, XY-mixer QAOA).
- **VeloxQ in action** — find a compelling application of VeloxQ within the manufacturing/scheduling domain, formulate and solve it as a QUBO, and demonstrate solution quality, scalability, computational performance, and insight into where VeloxQ works best.
- **Algorithm comparison** — QAOA vs VQE vs quantum annealing vs recursive QAOA; error mitigation (e.g., zero-noise extrapolation); warm-starting quantum parameters from a classical heuristic or LP relaxation for faster convergence.
- **Scaling curve & variance analysis** — how solution quality degrades as qubit count grows (e.g. 20 → 30 → 40), compute time vs. instance size; multiple runs with different seeds/parameters and proper statistics instead of a single run.
- **Due dates & tardiness** — the objective is no longer just overall makespan but minimizing lateness against per-job deadlines.
- **Sequence-dependent setup times** — switching a machine from one part type to another costs extra time; producing the same part type in a row incurs no setup.
- **Disruptions / anomalies** — inject perturbations (e.g. a machine outage) before planning and account for them, ideally statistically/robustly.
- **Rolling / partial-state planning** — plan from a state where some jobs are already partly processed and machines are mid-operation.
- **Functional scaling** — more machines, longer operation chains, and a cost-benefit analysis of buying an additional machine (“MetalTech is considering a second milling machine — use your model to justify whether it pays off”).

What a strong extension shows:

- **One integrated result.** Your reduction, algorithm comparison and scaling analysis are carried out on your extended model, not only on the baseline 3×3.
- **Verified correctness.** Every result is checked against a classical reference (e.g. CP-SAT) on the same model. If a smaller formulation loses the optimum, state the gap.
- **The price of realism.** For each constraint you add, report how many variables it costs in the naive time-indexed formulation and in yours.
- **A comparable number.** Also report your smallest correct QUBO for the baseline 3×3 instance (reference: 72 variables in the plain time-indexed formulation at H = 8; simple start-window pruning already gives 39), so teams with different models can be compared.
- **Depth over count.** One constraint fully formulated, solved and verified is worth more than three that are sketched.

A polished engineering and presentation layer (interactive Gantt chart, end-to-end pipeline) earns credit when it showcases a strong extension, but it cannot stand in for one.

6. What You Must Deliver

Submit all of the following by **Sunday 18.10, 15:00 UTC+2**.

1. **Code repository (Git)** containing the reference solution for Instance A, the reference solution for Instance B, and your Stage 2 extension, clearly separated and documented. The repository must be public (MIT license).
2. **Development log** (see Section 7) — a dated Markdown file in the repository.
3. **Project design** (see Section 8) in the repository.
4. **Demo video** (see Section 9) — a link (unlisted is fine).
5. README file with instructions on how to run your code.
6. Schedule files in the output format of Section 14 (schedule_2x2.json and schedule_3x3.json) for both baseline instances - they should also be included in the repository.

Then, attend the scheduled Q&A slot with the jury to present the **Live demo** (see Section 10). Q&A slots will be assigned on 18.10; each slot is 3 minutes of presentation plus ~2 minutes of questions.

A team is assessed further only if the Stage 1 baseline works for both instances.

Logistics: submit via Google Form (a link will be provided by the organizers). Teams have 1-5 members.

7. Development Log - required

As we want to understand your development process, each team should keep a Development Log: a dated record of how the project came to be. It must contain dated entries documenting, for example:

- How the project developed step by step (what you did and when)?
- What problems you hit and how you solved them?
- What design decisions you made and why (e.g. penalty weights, choice of encoding, choice of algorithm)?
- What you rejected and why?

The log takes the form of a running report (a Markdown file in the repo). **Consistency between the log, the actual code, and the presentation is a part of the score.**

8. Project Design - required

As we want to have a good understanding of your solution and evaluate your development process reliably, each team should also keep a project design. In this document, you should provide a precise explanation of the data model adopted for optimization and of the optimization methods used. In particular, you can include the following information:

- What exactly is the optimization problem?
- What does a state represent, what are the problem variables and parameters (e.g., the time horizon), and why were they chosen?

- What are the constraints, and why? How do the variables interact with each other, which ones are dependent or independent, and are there any weights?
- How and why did you model the above problem as that particular QUBO? What is the objective function, with its linear and quadratic terms, and what do those terms represent?
- What assumptions or simplifications does the model make?
- Is the problem feasible, and what is the feasibility condition?

9. Demo Video (up to 5 minutes) — required

At the end, each team records a short demo video that:

- Shows the program running live (launch, result),
- Walks through the key parts of the code — especially your own solutions and the places where you went beyond the baseline; **in particular, explain in detail the data model adopted for optimization and the optimization method,**
- Summary of the Development log.

10. Live Presentation (up to 3 minutes) — required

In addition to the Development Log, Project Design and Demo Video, each team presents their projects online (3 minutes + Q&A) and answers **jury questions** after the presentation. The presentations will start on 18.10 at 18:00 UTC+2.

The live demo should include:

- Presentation of the program running live (launch, result),
- Explanation of the most important design decisions (included in the Project Design) and conclusions.

11. Evaluation Criteria

The baseline (Section 4) is the required minimum — solving it correctly is only the starting point and the solutions that do not solve the baseline correctly will not be evaluated further. Later, the creativity, depth of analysis, and the quality of the Development log, Project Design, and Demo video matter just as much. Teams must go further (Section 5) to earn points for extending the solution.

- (0-5 points) Correctness of the problem formulation (QUBO) for the quantum implementation.
- (0-5 points) Quality of the solution (makespan vs. the reference / classical baseline).
- (0-5 points) Creativity and complexity of the chosen extension beyond the minimum (Section 5).
- (0-3 points) Quality of the Development log — clarity of reasoning, consistency with the code, justification of design decisions.
- (0-3 points) Quality of the Project Design.
- (0-3 points) Quality of the Demo video — understanding of your own solution and clarity in walking through the key parts of the code.
- (0-3 points) Quality of the Live presentation and answering questions.
- (0-3 points) Consistency between the Development log, the actual code, and the presentation.

For submissions using VeloxQ, the jury will assess the elegance and correctness of the integration with the VeloxQ backend, the quality of the obtained solutions, scalability with increasing problem size or complexity, the depth of insights derived from the experiments and from the use of VeloxQ, and the clarity and reproducibility of the overall approach.

12. Rules: Using AI

Using AI tools (coding assistants, language models, etc.) is **fully allowed and encouraged** — we treat them as a natural starting point, shared by all teams. We do not assess whether you used AI, only what you built on top of it: creativity, your own ideas, and developing the project along your own vision. Producing a working minimum is just the start — the result is decided by how far you take it along your own path.

Understand your solution to the core. Working with AI is a natural part of the process — but you must be able to explain your solution and take full responsibility for it. As you investigate and learn how it works, document that in the Development Log too.

We also require you to add a description of how AI was used in the project to the README.

13. Timeline

The event runs over one weekend

- **Day 1 Friday 16.10 (18:00 UTC+2 - kickoff):** form the team and lock it in (#lock-in-teams channel) and start working on your project.
- **Days 2-3 Saturday / Sunday (till 15:00 UTC+2):** complete Instances A and B, develop the extension and submit your solution.
- **Day 3 Sunday 18:00 UTC+2:** live presentations

14. Input and Output Data Format (JSON)

Instances (jobs, machines, operation times) are provided as JSON. The same schema describes both instances; they differ only in the contents of jobs. This applies to the base problem in Section 4. For the final solution, the JSON schema can be extended or changed to cover different real-life scenarios in a factory.

Instance A — instance_2x2.json:

```
{
  "problem": "erp_scheduling",
  "jobs": [
    { "id": "Z1", "operations": [
        { "machine": "T", "duration": 2 },
        { "machine": "F", "duration": 1 } ] },
    { "id": "Z2", "operations": [
        { "machine": "T", "duration": 1 },
        { "machine": "F", "duration": 2 } ] }
  ]
}
```

Instance B — instance_3x3.json:

```
{
  "problem": "erp_scheduling",
  "jobs": [
    { "id": "Z1", "operations": [
```

```
        { "machine": "T", "duration": 2 },
        { "machine": "F", "duration": 1 },
        { "machine": "S", "duration": 2 } ] },
    { "id": "Z2", "operations": [
        { "machine": "T", "duration": 1 },
        { "machine": "F", "duration": 2 },
        { "machine": "S", "duration": 1 } ] },
    { "id": "Z3", "operations": [
        { "machine": "T", "duration": 2 },
        { "machine": "F", "duration": 2 },
        { "machine": "S", "duration": 1 } ] }
  ]
}
```

- **Machine order:** in the baseline you may hard-code the global technological order T → F → S (T → F for Instance A). Handling a per-job or configurable machine order is a natural Stage-2 extension.
- **Output format:** for both baseline instances, write the resulting schedule as JSON in the format below, so the jury can check feasibility and makespan automatically. Times are integers in hours starting at 0, and end = start + duration. Example for Instance A:

```
{
  "instance": "instance_2x2",
  "method": "QAOA (state-vector simulator)",
  "makespan": 4,
  "schedule": [
    { "job": "Z2", "machine": "T", "start": 0, "end": 1 },
    { "job": "Z2", "machine": "F", "start": 1, "end": 3 },
    { "job": "Z1", "machine": "T", "start": 1, "end": 3 },
    { "job": "Z1", "machine": "F", "start": 3, "end": 4 }
  ]
}
```

15. Prizes

Two prizes will be awarded at the end of the hackathon:

- **2000 PLN — Overall winner.** Awarded to the team with the strongest overall submission, judged across all the criteria in the *Evaluation Criteria* section — the quality of the quantum formulation and solution, the depth and creativity of the extension, and the standard of the Development log, Project Design, demo video, and live defense.
- **1000 PLN — Best use of VeloxQ.** Awarded to the team that integrates VeloxQ most effectively into an end-to-end solution, demonstrating a compelling use of the solver and showing how it can complement classical and quantum approaches within a broader optimization workflow.

16. VeloxQ Access Token

Once the hackathon has started and teams have been formed (declared in the #lock-in-teams channel on the hackathon's Discord server), one designated representative from each team should send an email to support@veloxq.com (and add contact@qaif.org in CC) with the subject line: "QPoland QUANTUM HACKATHON 2026".

The request should be sent from the email address that the team intends to use for communication regarding VeloxQ access. In response, the team representative will receive an individual access token for connecting to the VeloxQ backend, together with the technical information and instructions required to establish the connection and start using the service.

Teams are encouraged to request access as soon as possible after team formation. The access token should be treated as confidential and shared only within the team.